1. Importazione delle librerie

python

from scapy.all import sniff, IP, TCP, UDP

Questa riga importa quattro elementi dalla libreria Scapy.

- sniff: permette di catturare pacchetti di rete.
- IP: permette di identificare e leggere il livello IP di un pacchetto.
- TCP: permette di identificare i pacchetti che utilizzano il protocollo TCP.
- UDP: permette di identificare i pacchetti che utilizzano il protocollo UDP.

Questi elementi saranno utilizzati per intercettare il traffico visibile all'interfaccia di rete e analizzarne alcune informazioni.

Per installare Scapy, se non è già presente, puoi utilizzare:

bash

pip install scapy

La cattura potrebbe richiedere privilegi amministrativi, a seconda del sistema operativo.

2. Definizione della funzione

python

def analizza_pacchetto(pacchetto):

Questa riga definisce una funzione chiamata analizza_pacchetto.

- def è la parola chiave che serve a definire una funzione.
- analizza_pacchetto è il nome della funzione.
- pacchetto è il parametro che riceverà il pacchetto catturato.

La funzione contiene le istruzioni necessarie per analizzare un pacchetto e stampare le informazioni che interessano.

Da sola, però, la definizione non avvia alcuna cattura. La funzione verrà richiamata successivamente da sniff().

3. Verifica della presenza del livello IP

python

if IP in pacchetto:

Questa istruzione controlla se il pacchetto contiene un livello IP riconosciuto da Scapy.

- if significa “se”.
- IP in pacchetto verifica la presenza del livello IP.

Se il livello IP è presente, il programma esegue il blocco indentato successivo.

Se non è presente, salta quel blocco e la funzione termina senza stampare informazioni.

4. Lettura dell'indirizzo IP sorgente

python

sorgente = pacchetto[IP].src

Questa riga legge l'indirizzo IP sorgente e lo salva nella variabile sorgente.

- pacchetto[IP] accede al livello IP.
- .src restituisce l'indirizzo sorgente.

Per esempio, il valore potrebbe essere:

text

192.168.1.25

Questo è l'indirizzo sorgente indicato nel pacchetto.

5. Lettura dell'indirizzo IP destinazione

python

destinazione = pacchetto[IP].dst

Legge l'indirizzo IP di destinazione e lo memorizza nella variabile destinazione.

- .dst significa *destination*, cioè destinazione.

Per esempio:

text

142.250.184.14

Questo è un indirizzo esemplificativo: il valore reale dipende dal traffico catturato.

6. Lettura del protocollo IP

python

protocollo = pacchetto[IP].proto

Questa riga legge il numero identificativo del protocollo trasportato nel pacchetto IP.

Alcuni valori comuni sono:

Numero	Protocollo
1	ICMP
6	TCP
17	UDP

Il risultato viene memorizzato nella variabile protocollo.

È un valore numerico, non il nome testuale del protocollo.

7. Separazione visiva dei risultati

python

print("-" * 60)

Questa istruzione stampa 60 trattini.

- print() mostra il risultato sullo schermo.
- "-" è la stringa da stampare.
- * 60 ripete la stringa 60 volte.

Serve a separare le informazioni di un pacchetto da quelle del successivo.

8. Stampa dell'IP sorgente

python

print(f"IP sorgente: {sorgente}")

Visualizza l'indirizzo IP sorgente.

La lettera f prima della stringa indica una *f-string*: permette di inserire il valore di una variabile all'interno del testo usando le parentesi graffe.

Per esempio:

text

IP sorgente: 192.168.1.25

9. Stampa dell'IP destinazione

python

print(f"IP destinazione: {destinazione}")

Visualizza l'indirizzo IP di destinazione.

Un possibile risultato è:

text

IP destinazione: 142.250.184.14

Le righe precedenti mostrano quindi da quale IP proviene il pacchetto e verso quale IP è diretto.

10. Verifica del protocollo TCP

python

if TCP in pacchetto:

Questa condizione verifica se il pacchetto contiene un livello TCP.

Se è presente, il programma entra nel blocco e stampa le informazioni TCP.

python

print("Protocollo: TCP")

Mostra il nome del protocollo.

python

print(f"Porta sorgente: {pacchetto[TCP].sport}")

Legge e stampa la porta TCP sorgente.

- pacchetto[TCP] accede al livello TCP.
- .sport indica la porta sorgente.

python

print(f"Porta destinazione: {pacchetto[TCP].dport}")

Legge e stampa la porta TCP di destinazione.

- .dport indica la porta di destinazione.

Un esempio di risultato potrebbe essere:

text

Protocollo: TCP

Porta sorgente: 51524

Porta destinazione: 443

In questo esempio, il pacchetto usa la porta 51524 come sorgente e la porta 443 come destinazione.

11. Verifica del protocollo UDP

python

elif UDP in pacchetto:

elif significa “altrimenti, se”.

Questa condizione viene controllata se il pacchetto non è stato riconosciuto come TCP dal controllo precedente.

Se è presente UDP, vengono eseguite le istruzioni successive.

python

print("Protocollo: UDP")

Stampa il nome del protocollo UDP.

python

print(f"Porta sorgente: {pacchetto[UDP].sport}")

Legge e stampa la porta UDP sorgente.

python

print(f"Porta destinazione: {pacchetto[UDP].dport}")

Legge e stampa la porta UDP di destinazione.

Per esempio:

text

Protocollo: UDP

Porta sorgente: 53000

Porta destinazione: 53

La porta 53 è comunemente utilizzata dal servizio DNS.

12. Gestione degli altri protocolli

python

else:

  print(f"Protocollo IP: {protocollo}")

Questo else viene eseguito quando il pacchetto contiene IP, ma non è stato riconosciuto come TCP o UDP.

Il programma stampa il numero del protocollo IP letto in precedenza.

Per esempio, per un pacchetto ICMP potrebbe mostrare:

text

Protocollo IP: 1

Questo consente di mostrare un'informazione anche per pacchetti IP che non contengono TCP o UDP.

13. Messaggi iniziali

python

print("Cattura del traffico avviata...")

Stampa un messaggio che informa l'utente dell'avvio della cattura.

python

print("Premi CTRL+C per interrompere.")

Indica come interrompere manualmente il programma.

Premendo CTRL+C, normalmente Python genera un'eccezione chiamata KeyboardInterrupt.

14. Blocco try

python

try:

try apre un blocco in cui il programma prova a eseguire le istruzioni successive.

Nel tuo codice serve soprattutto a gestire l'interruzione manuale della cattura tramite il blocco except.

15. Avvio della cattura con sniff()

python

```
sniff(
    filter="tcp port 80 or tcp port 443 or udp",
    prn=analizza_pacchetto,
    store=False
)
```

Questa è la parte centrale del programma: avvia la cattura dei pacchetti.

Il parametro filter

python

filter="tcp port 80 or tcp port 443 or udp",

Specifica quali pacchetti devono essere selezionati.

- tcp port 80: pacchetti TCP associati alla porta 80.
- or: significa “oppure”.
- tcp port 443: pacchetti TCP associati alla porta 443.
- or udp: include il traffico UDP, senza limitare il filtro a una specifica porta UDP.

In pratica, il filtro seleziona il traffico TCP relativo alle porte 80 e 443, oltre ai pacchetti UDP.

Le porte 80 e 443 sono comunemente associate rispettivamente a HTTP e HTTPS. Tuttavia, il filtro identifica le porte TCP, non verifica che il contenuto sia effettivamente HTTP o HTTPS.

Il parametro prn

python

prn=analizza_pacchetto,

Questa è la modifica importante rispetto alla versione precedente.

Il parametro prn indica quale funzione deve essere eseguita per ogni pacchetto catturato.

In questo caso:

- sniff() riceve un pacchetto.
- Passa quel pacchetto alla funzione analizza_pacchetto().
- La funzione legge gli indirizzi IP, controlla il protocollo e stampa le porte quando disponibili.

Non servono parentesi dopo analizza_pacchetto, perché stai passando il riferimento alla funzione, non chiamandola immediatamente.

Il parametro store

python

store=False

Indica a Scapy di non conservare in memoria tutti i pacchetti catturati.

Questo è utile perché la cattura può continuare a lungo: i pacchetti vengono elaborati man mano, senza accumularli tutti in una lista.

16. Gestione dell'interruzione

python

except KeyboardInterrupt:

Intercetta l'interruzione manuale, normalmente generata premendo CTRL+C.

python

print("\nCattura terminata.")

Stampa un messaggio finale.

Il carattere \n inserisce una nuova riga prima del testo.

In questo modo, quando interrompi il programma, viene visualizzato un messaggio comprensibile anziché lasciare la terminazione senza indicazioni.

Esempio di output

Se viene catturato un pacchetto TCP diretto alla porta 443, il programma potrebbe mostrare:

text

Cattura del traffico avviata...

Premi CTRL+C per interrompere.

IP sorgente: 192.168.1.25

IP destinazione: 142.250.184.14

Protocollo: TCP

Porta sorgente: 51524

Porta destinazione: 443

Gli indirizzi e le porte sono soltanto esemplificativi.

Come funziona il programma nel complesso

1. Importa Scapy

2. Definisce la funzione di analisi

3. Avvia la cattura con sniff()

4. Filtra il traffico TCP sulle porte 80 e 443 e il traffico UDP

5. Passa ogni pacchetto a analizza_pacchetto()

6. Stampa IP, protocollo e porte disponibili

7. Termina con CTRL+C